

Tabular data

A particular type of data structure



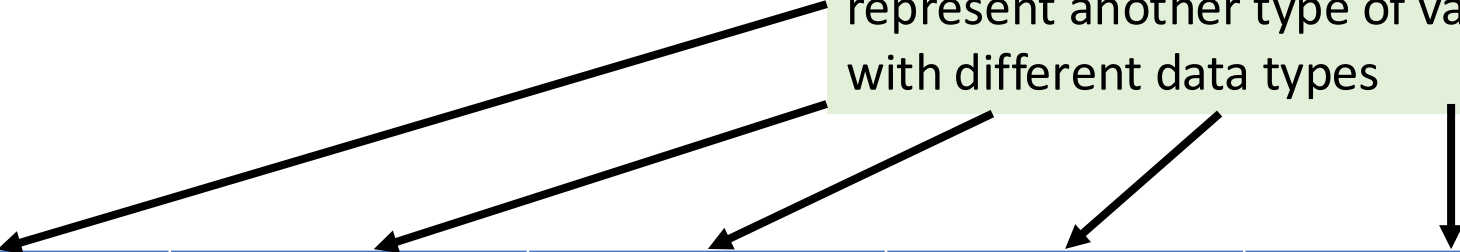
Spreadsheets and databases rule the world!



Ariel Fischman holds the Guinness World Record for owning the most spreadsheet software (over 500!)

What is tabular data?

Unlike arrays, each column can represent another type of value, with different data types



Date (index)	Wind speed	Wind direction	Rain fall (mm)	Hours of sun
7.3.2024	7.1	N	0.0	10
8.3.2024	0.3	NW	2.1	2
9.3.2024	1.1	SE	0.3	5

What is tabular data?

Column and rows have meaningful labels (indices) that are attached to the data for each operation

Date (index)	Wind speed	Wind direction	Rain fall (mm)	Hours of sun
7.3.2024	7.1	N	0.0	10
8.3.2024	0.3	NW	2.1	2
9.3.2024	1.1	SE	0.3	5

What is tabular data?

Date (index)	Wind speed	Wind direction	Rain fall (mm)	Hours of sun
7.3.2024	7.1	N	0.0	10
8.3.2024	0.3	NW	2.1	2
9.3.2024	1.1	SE	0.3	5

Subject ID (index)	Condition ID	Date	Response time (ms)	Response
VM	732	7.3.2024	28	LEFT
VM	567	8.3.2024	41	RIGHT
PB	732	9.3.2024	73	LEFT

What is tabular data?

Does the windspeed affect response time?

Date (index)	Wind speed	Wind direction	Rain fall (mm)	Hours of sun
7.3.2024	7.1	N	0.0	10
8.3.2024	0.3	NW	2.1	2
9.3.2024	1.1	SE	0.3	5

Subject ID (index)	Condition ID	Date	Response time (ms)	Response
VM	732	7.3.2024	28	LEFT
VM	567	8.3.2024	41	RIGHT
PB	732	9.3.2024	73	LEFT

Many tools to handle tabular data

- Python tools
 - pandas: in-memory tabular data
 - dask: on-disk tabular data
- SQL databases
 - Optimized for retrieving rows (tree data structure for index)
 - Transactional: groups of operations are either all executed, or none
- Columnar DBs, Spark, Hadoop
 - Optimized for operations on columns
 - Ideal for data science tasks
 - Operations can be automatically distributed over multiple machines

Many tools to handle tabular data

- Python tools

- pandas
- dask

- SQL

- Oracle
- Teradata

- Columnar

- Oracle
- Informatica
- Vertica

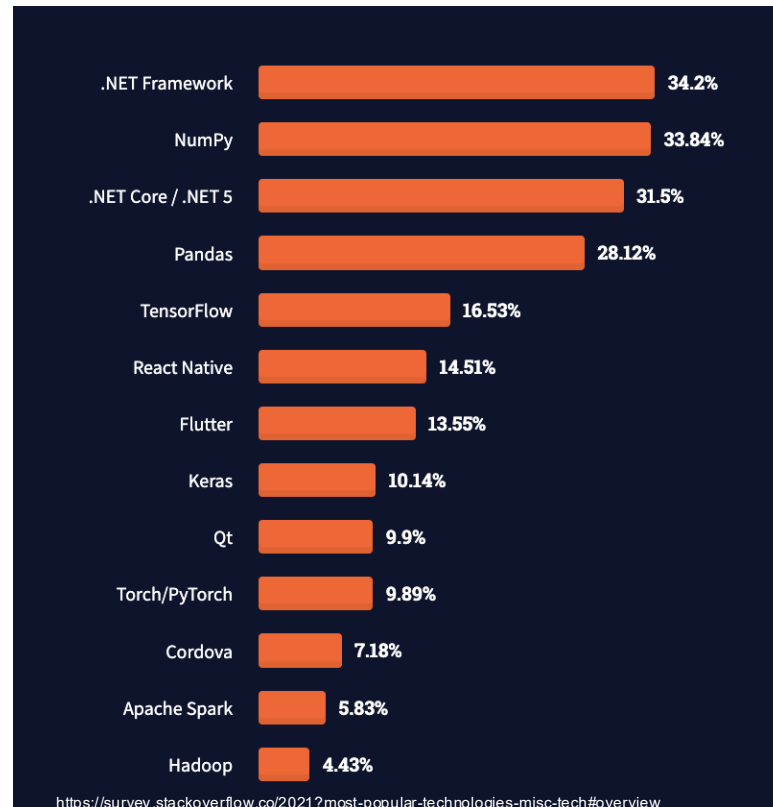
Today we will see an example in *pandas*,
but the reasoning we will see are general
and apply to all of them



Why pandas?

Stack overflow survey 2021

Which **other frameworks and libraries** have you done extensive development work in over the past year, and which do you want to work in over the next year?

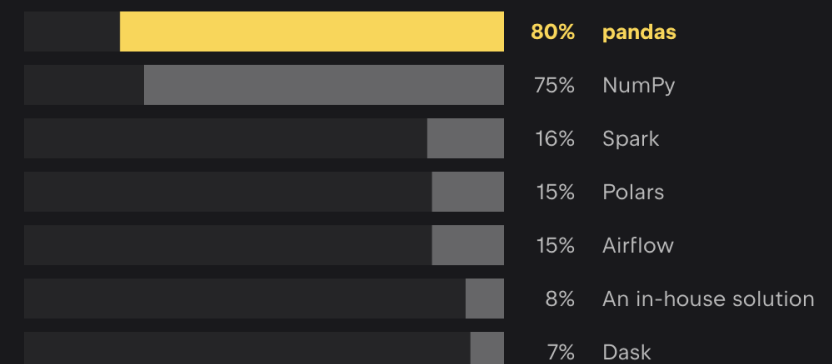


Data Science

51%

of all surveyed Python developers are involved in data exploration and processing, with pandas and NumPy being the tools mostly used for it.

Tools for data exploration and processing



<https://lp.jetbrains.com/python-developers-survey-2024/>

Hands-on

- Create a single table combining the information from the two separate tabular data files

```
predimed_records.csv  
    &  
predimed_locations.csv
```

Exercise 20 minutes



```
exercises/tabular_join/  
tabular_join.ipynb
```

Hands-on

- Create a single table combining the information from the two separate tabular data files

```
predimed_records.csv  
    &  
predimed_locations.csv
```

- Extra: what happens if we increase the table size? How is the performance of your solution affected?

Exercise 20 minutes



```
exercises/tabular_join/  
tabular_join.ipynb
```

Tabular data join and performance

predimed_records

6324 (N) rows
16 columns

	patient-id	location-id	sex	age	smoke	bmi	waist	wth	htn	diab	hyperchol	famhist	hormo	p14	toevent	event
0	436	4	Male	58	Former	33.53	122	0.753086	No	No	Yes	No	No	10	5.374401	Yes
1	1130	4	Male	77	Current	31.05	119	0.730061	Yes	Yes	No	No	No	10	6.097194	No
2	1131	4	Female	72	Former	30.86	106	0.654321	No	Yes	No	Yes	No	8	5.946612	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6323	1257	5	Male	58	Former	24.43	93	0.547059	Yes	Yes	Yes	No	No	9	2.590007	No

6324 rows x 16 columns

predimed_locations

5 (M) rows
3 columns

	location-id	city
0	4	Madrid
1	1	Valencia
2	2	Barcelona
3	5	Bilbao
4	3	Malaga

Solution

predimed_records.csv

N rows

	patient-id	location-id	sex	age
0	436	4	Male	58
1	1130	4	Male	77
2	1131	4	Female	72
...	...	...	...	...
6323	1257	5	Male	58
6324 rows x 16 columns				

predimed_locations.csv

M rows

	location-id	city
0	4	Madrid
1	1	Valencia
2	2	Barcelona
3	5	Bilbao
4	3	Malaga

Solution

```
patients_with_city = df_patients.copy()
patients_with_city['city_2_for_loops'] = 'n/a'

for idx, row in patients_with_city.iterrows():
    location = row['location-id']
    for loc in df_locations['location-id']:
        if loc == location:
            city = df_locations['city'][df_locations['location-id'] == loc].values
    if len(city) > 0:
        patients_with_city.loc[idx, 'city'] = city[0]
```

N rows

predimed_records.csv

	patient-id	location-id	sex	age
0	436	4	Male	58
1	1130	4	Male	77
2	1131	4	Female	72
...	...	...	...	...
6323	1257	5	Male	58
6324 rows x 16 columns				

predimed_locations.csv

M rows

	location-id	city
0	4	Madrid
1	1	Valencia
2	2	Barcelona
3	5	Bilbao
4	3	Malaga

Solution

```
patients_with_city = df_patients.copy()
patients_with_city['city_2_for_loops'] = 'n/a'

for idx, row in patients_with_city.iterrows(): O(N)
    location = row['location-id']
    for loc in df_locations['location-id']: O(M)
        if loc == location:
            city = df_locations['city'][df_locations['location-id'] == loc].values
    if len(city) > 0:
        patients_with_city.loc[idx, 'city'] = city[0]
```

predimed_records.csv

N rows

	patient-id	location-id	sex	age
0	436	4	Male	58
1	1130	4	Male	77
2	1131	4	Female	72
...	...	...	...	...
6323	1257	5	Male	58

6324 rows x 16 columns

predimed_locations.csv

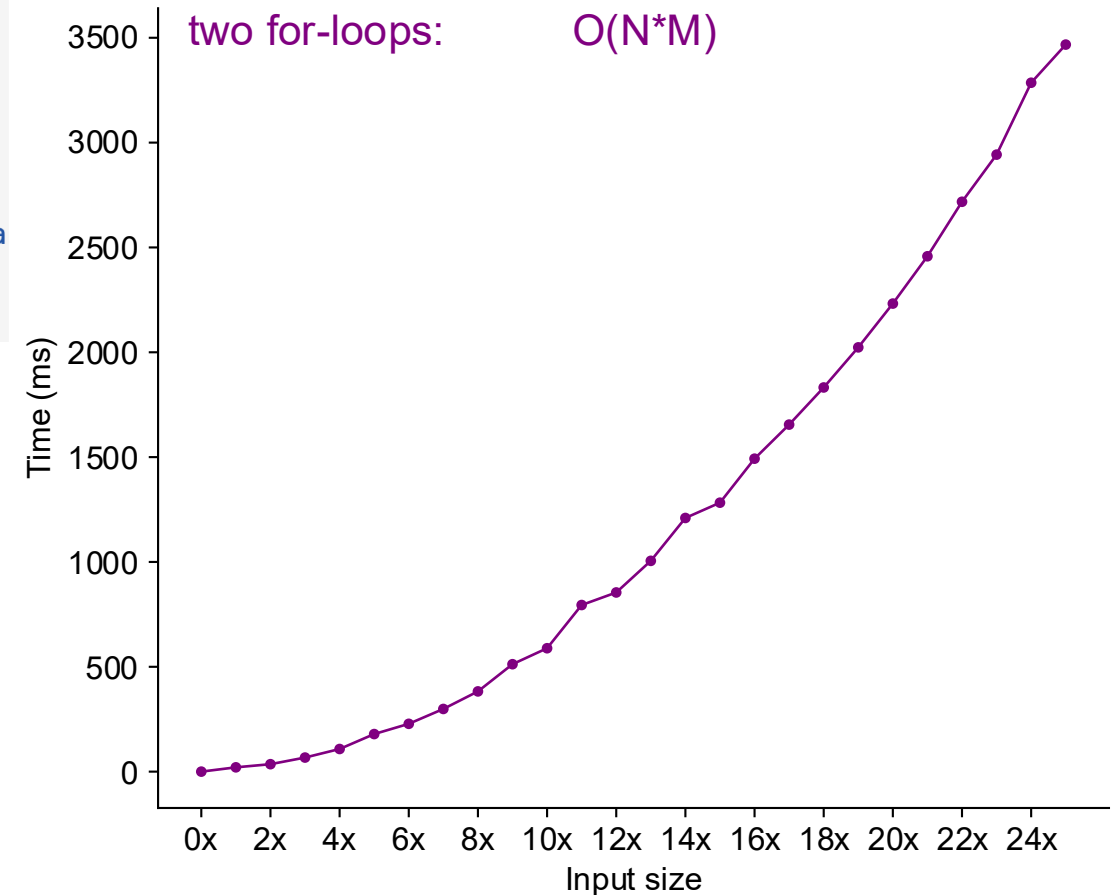
M rows

	location-id	city
0	4	Madrid
1	1	Valencia
2	2	Barcelona
3	5	Bilbao
4	3	Malaga

Solution $O(N*M)$

```
patients_with_city = df_patients.copy()
patients_with_city['city_2_for_loops'] = 'n/a'

for idx, row in patients_with_city.iterrows():  $O(N)$ 
    location = row['location-id']
    for loc in df_locations['location-id']:  $O(M)$ 
        if loc == location:
            city = df_locations['city'][df_locations['location-id'] == loc].values
    if len(city) > 0:
        patients_with_city.loc[idx, 'city'] = city[0]
```



Solution sort

predimed_records.csv

N rows

	patient-id	location-id	sex	age
0	436	4	Male	58
1	1130	4	Male	77
2	1131	4	Female	72
...	...	...	...	...
6323	1257	5	Male	58

6324 rows x 16 columns

N rows

	patient-id	location-id	sex	age
5219	1	1	Female	77
1153	2	1	Female	68
5731	3	1	Female	66
...	...	...	...	...
6323	1257	5	Male	58

6324 rows x 19 columns

sort on
location-id

predimed_locations.csv

M rows

	location-id	city
0	4	Madrid
1	1	Valencia
2	2	Barcelona
3	5	Bilbao
4	3	Malaga

M rows

	location-id	city
1	1	Valencia
2	2	Barcelona
4	3	Malaga
0	4	Madrid
3	5	Bilbao

Solution sort

```
patients_with_city['city_with_sort'] = 'n/a'
```

```
sorted_patients = patients_with_city.sort_values(['location-id'])
```

```
sorted_locations = df_locations.sort_values(['location-id'])
```

$O(N \log N)$

$O(M \log M)$

N rows

predimed_records.csv

	patient-id	location-id	sex	age
5219	1	1	Female	77
1153	2	1	Female	68
5731	3	1	Female	66
...	...	...	...	...
6323	1257	5	Male	58

6324 rows × 19 columns

predimed_locations.csv

	location-id	city
1	1	Valencia
2	2	Barcelona
4	3	Malaga
0	4	Madrid
3	5	Bilbao

M rows

Solution sort

```
patients_with_city['city_with_sort'] = 'n/a'
```

```
sorted_patients = patients_with_city.sort_values(['location-id'])
sorted_locations = df_locations.sort_values(['location-id'])
```

```
city_2_col = sorted_patients.columns.get_loc('city_with_sort')
locations_idx = 0
patients_idx = 0
```

while True: $O(N + M)$

```
    row_locations = sorted_locations.iloc[locations_idx]
    key_locations = row_locations['location-id']
```

```
    row_patients = sorted_patients.iloc[patients_idx]
    key_patients = row_patients['location-id']
```

```
    if key_patients == key_locations:
        original_idx = sorted_patients.index[patients_idx]
        patients_with_city.iloc[original_idx, city_2_col] = row_locations['city']
        patients_idx += 1
```

```
    else:
        locations_idx += 1
        if locations_idx >= len(sorted_locations):
            break
```

```
    if patients_idx >= len(sorted_patients):
        break
```

$O(N \log N)$

$O(M \log M)$

N rows

predimed_records.csv

	patient-id	location-id	sex	age
5219	1	1	Female	77
1153	2	1	Female	68
5731	3	1	Female	66
...	...	...	...	...
6323	1257	5	Male	58

6324 rows × 19 columns

predimed_locations.csv

	location-id	city
1	1	Valencia
2	2	Barcelona
4	3	Malaga
0	4	Madrid
3	5	Bilbao

M rows

Solution sort

```
patients_with_city['city_with_sort'] = 'n/a'
```

```
sorted_patients = patients_with_city.sort_values(['location-id'])
sorted_locations = df_locations.sort_values(['location-id'])
```

```
city_2_col = sorted_patients.columns.get_loc('city_with_sort')
locations_idx = 0
patients_idx = 0
```

```
while True:
    row_locations = sorted_locations.iloc[locations_idx]
    key_locations = row_locations['location-id']

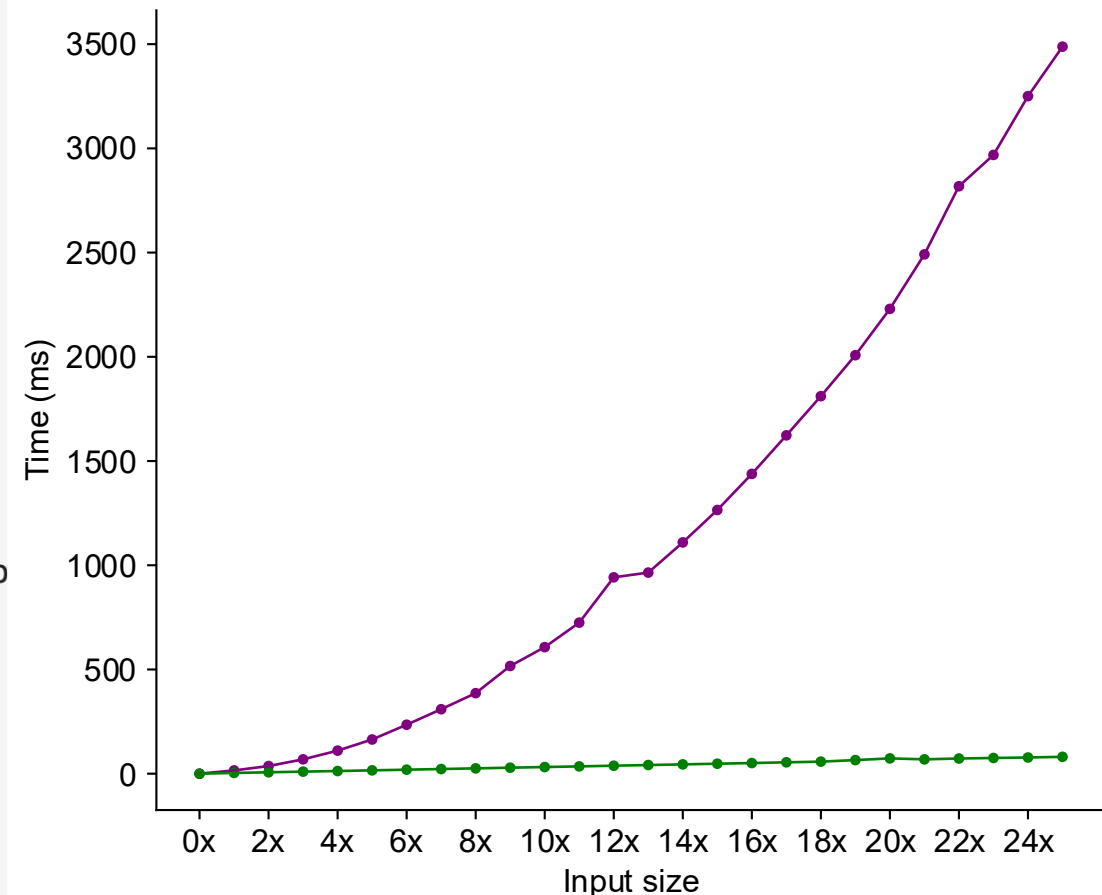
    row_patients = sorted_patients.iloc[patients_idx]
    key_patients = row_patients['location-id']

    if key_patients == key_locations:
        original_idx = sorted_patients.index[patients_idx]
        patients_with_city.iloc[original_idx, city_2_col] = row_locations['city_with_sort']
        patients_idx += 1
    else:
        locations_idx += 1
        if locations_idx >= len(sorted_locations):
            break
    if patients_idx >= len(sorted_patients):
        break
```

$O(N \log N)$
 $O(M \log M)$

two for-loops:
sorted for loop:

$O(N \cdot M)$
 $O(N \log N + M \log M)$



Solution join operation

```
df_patients_with_city = df_patients.merge(df_locations,  
                                           on=['location-id'],  
                                           how='left')
```

	patient-id	location-id	sex	age	smoke	bmi	waist	wth	htn	diab	hyperchol	famhist	hormo	p14	toevent	event	city
0	436	4	Male	58	Former	33.53	122	0.753086	No	No	Yes	No	No	10	5.374401	Yes	Madrid
1	1130	4	Male	77	Current	31.05	119	0.730061	Yes	Yes	No	No	No	10	6.097194	No	Madrid
2	1131	4	Female	72	Former	30.86	106	0.654321	No	Yes	No	Yes	No	8	5.946612	No	Madrid
3	1132	4	Male	71	Former	27.68	118	0.694118	Yes	No	Yes	No	No	8	2.907598	Yes	Madrid
4	1111	2	Female	79	Never	35.94	129	0.806250	Yes	No	Yes	No	No	9	4.761123	No	Barcelona
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6319	120	5	Female	66	Never	28.51	104	0.645963	Yes	No	Yes	Yes	No	8	3.550992	No	Bilbao
6320	118	5	Male	80	Never	23.81	109	0.589189	Yes	Yes	Yes	Yes	No	8	2.743326	No	Bilbao
6321	351	3	Male	57	Former	25.24	100	0.571429	Yes	No	Yes	No	NaN	7	0.479124	No	Malaga
6322	499	5	Female	71	Never	32.04	98	0.653333	Yes	No	Yes	Yes	No	6	2.587269	No	Bilbao
6323	1257	5	Male	58	Former	24.43	93	0.547059	Yes	Yes	Yes	No	No	9	2.590007	No	Bilbao

6324 rows x 17 columns

Solution join operation

```
df_patients_with_city = df_patients.merge(df_locations,  
                                          on=['location-id'],  
                                          how='left')
```

column or index level
names to join on,
found in both dataframes

type of merge to be
performed. Here the
keys and the order of
the left dataframe
`df_patients` are kept

	patient-id	location-id	sex	age	smoke	bmi	waist	wth	htn	diab	hyperchol	famhist	hormo	p14	toevent	event	city
0	436	4	Male	58	Former	33.53	122	0.753086	No	No	Yes	No	No	10	5.374401	Yes	Madrid
1	1130	4	Male	77	Current	31.05	119	0.730061	Yes	Yes	No	No	No	10	6.097194	No	Madrid
2	1131	4	Female	72	Former	30.86	106	0.654321	No	Yes	No	Yes	No	8	5.946612	No	Madrid
3	1132	4	Male	71	Former	27.68	118	0.694118	Yes	No	Yes	No	No	8	2.907598	Yes	Madrid
4	1111	2	Female	79	Never	35.94	129	0.806250	Yes	No	Yes	No	No	9	4.761123	No	Barcelona
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6319	120	5	Female	66	Never	28.51	104	0.645963	Yes	No	Yes	Yes	No	8	3.550992	No	Bilbao
6320	118	5	Male	80	Never	23.81	109	0.589189	Yes	Yes	Yes	Yes	No	8	2.743326	No	Bilbao
6321	351	3	Male	57	Former	25.24	100	0.571429	Yes	No	Yes	No	NaN	7	0.479124	No	Malaga
6322	499	5	Female	71	Never	32.04	98	0.653333	Yes	No	Yes	Yes	No	6	2.587269	No	Bilbao
6323	1257	5	Male	58	Former	24.43	93	0.547059	Yes	Yes	Yes	No	No	9	2.590007	No	Bilbao

6324 rows x 17 columns

Solution join operation

```
df_patients_with_city = df_patients.merge(df_locations,
                                           on=['location-id'],
                                           how='left')
```

	patient-id	location-id	sex	age	smoke	bmi	waist	wth	htn	diab	hyperchol	famhist	hormo	p14	1
0	436	4	Male	58	Former	33.53	122	0.753086	No	No	Yes	No	No	10	5.
1	1130	4	Male	77	Current	31.05	119	0.730061	Yes	Yes	No	No	No	10	6.
2	1131	4	Female	72	Former	30.86	106	0.654321	No	Yes	No	Yes	No	8	5.
3	1132	4	Male	71	Former	27.68	118	0.694118	Yes	No	Yes	No	No	8	2.
4	1111	2	Female	79	Never	35.94	129	0.806250	Yes	No	Yes	No	No	9	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6319	120	5	Female	66	Never	28.51	104	0.645963	Yes	No	Yes	Yes	No	8	3.
6320	118	5	Male	80	Never	23.81	109	0.589189	Yes	Yes	Yes	Yes	No	8	2.
6321	351	3	Male	57	Former	25.24	100	0.571429	Yes	No	Yes	No	NaN	7	0.
6322	499	5	Female	71	Never	32.04	98	0.653333	Yes	No	Yes	Yes	No	6	2.
6323	1257	5	Male	58	Former	24.43	93	0.547059	Yes	Yes	Yes	No	No	9	2.

6324 rows x 17 columns

two for-loops:

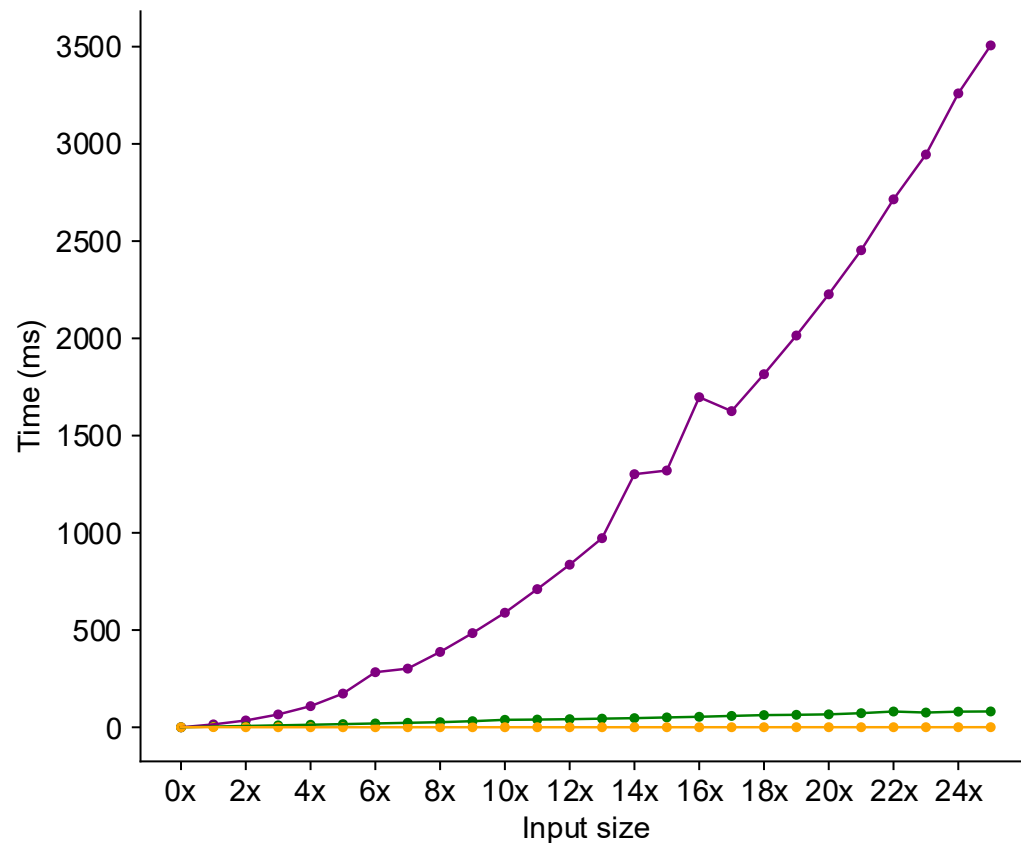
sorted for loop:

join operation:

$O(N \cdot M)$

$O(N \log N + M \log M)$

$O(N + M)$



- Combine information across tables (**join, anti-join**)

Common operations on tabular data

- **Join:** e.g. combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
- **Anti-join:** e.g. student compiles list of outliers, exclude them from the table of experiments to analyze

- Combine information across tables (**join, anti-join**)

Common operations on tabular data

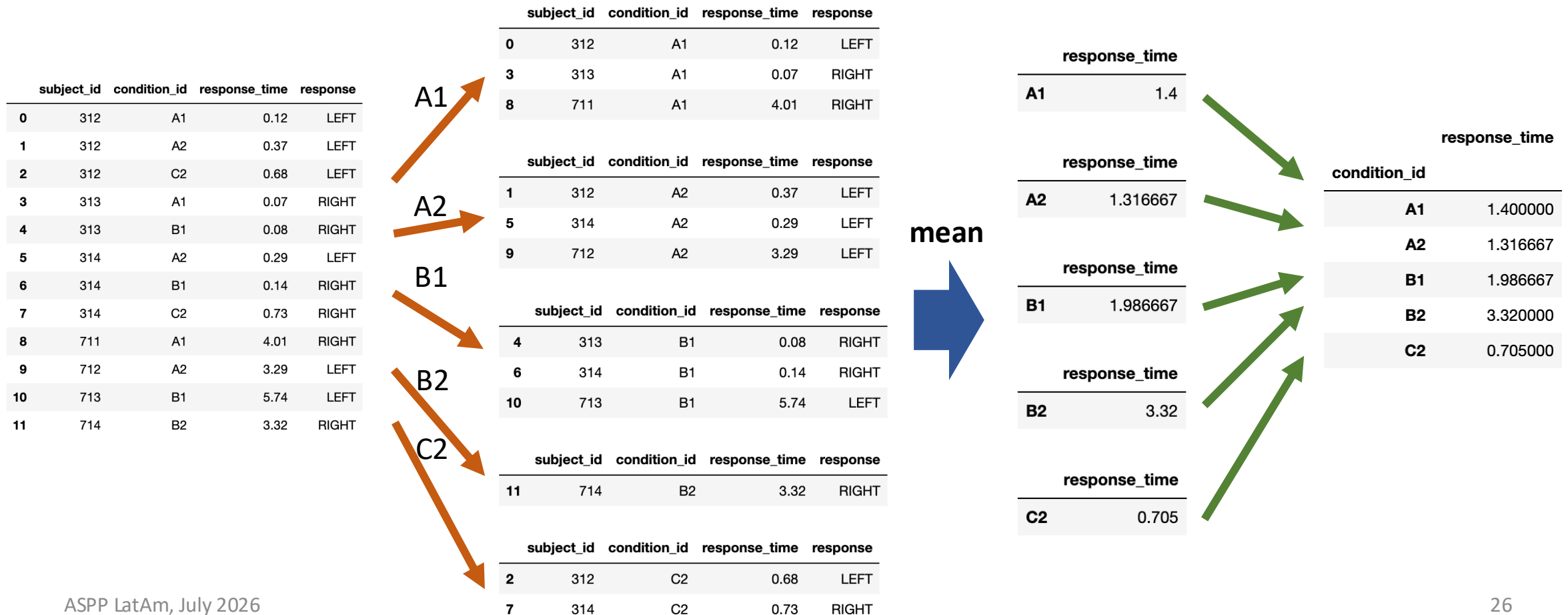
- **Join**: e.g. combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
 - **Anti-join**: e.g. student compiles list of outliers, exclude them from the table of experiments to analyze
- Summary tables (**split-apply-combine**)
 - E.g., compute average measurement and standard deviation by experimental condition and treatment dosage

The basic structure of most numerical analyses

split

apply

combine



```
df.groupby('condition_id')['response_time'].mean()
```

split

apply

combine

subject_id	condition_id	response_time	response
0	312	A1	0.12
1	312	A2	0.37
2	312	C2	0.68
3	313	A1	0.07
4	313	B1	0.08
5	314	A2	0.29
6	314	B1	0.14
7	314	C2	0.73
8	711	A1	4.01
9	712	A2	3.29
10	713	B1	5.74
11	714	B2	3.32

A1

A2

B1

B2

C2

subject_id	condition_id	response_time	response
0	312	A1	0.12
3	313	A1	0.07
8	711	A1	4.01

subject_id	condition_id	response_time	response
1	312	A2	0.37
5	314	A2	0.29
9	712	A2	3.29

subject_id	condition_id	response_time	response
4	313	B1	0.08
6	314	B1	0.14
10	713	B1	5.74

subject_id	condition_id	response_time	response
11	714	B2	3.32

subject_id	condition_id	response_time	response
2	312	C2	0.68
7	314	C2	0.73

mean

response_time
A1
1.4

response_time
A2
1.316667

response_time
B1
1.986667

response_time
B2
3.32

response_time
C2
0.705

condition_id	response_time
A1	1.400000
A2	1.316667
B1	1.986667
B2	3.320000
C2	0.705000

Split-apply-combine

- Tabular data tools vectorize the standard *split-apply-combine* operations, using a “group-by” command
- In addition, pandas has a “pivot-table” command that can be used to simplify the creation of more complex summary tables

```
data.pivot_table(
    index = 'condition_id', columns = 'response',
    values = 'response_time', aggfunc = 'mean',
)
```

split

apply

combine

	subject_id	condition_id	response_time	response
0	312	A1	0.12	LEFT
1	312	A2	0.37	LEFT
2	312	C2	0.68	LEFT
3	313	A1	0.07	RIGHT
4	313	B1	0.08	RIGHT
5	314	A2	0.29	LEFT
6	314	B1	0.14	RIGHT
7	314	C2	0.73	RIGHT
8	711	A1	4.01	RIGHT
9	712	A2	3.29	LEFT
10	713	B1	5.74	LEFT
11	714	B2	3.32	RIGHT



	response	LEFT	RIGHT
condition_id			
A1	0.12	2.04	
A2	1.32	NaN	
B1	5.74	0.11	
B2	NaN	3.32	
C2	0.68	0.73	

Hands-on

- Pt1: Work with the dataset from the previous exercise to answer the question: Is the mediterranean diet associated with less cardiovascular events?

Exercise pt 1: 15 minutes



```
exercises/tabular_split  
_apply_combine/split_ap  
ply_combine_part1.ipynb
```

Split-apply-combine operations and performance

	patient-id	group	event	event_int
0	436	Control	Yes	1
1	1130	Control	No	0
2	1131	MedDiet + VOO	No	0
3	1132	MedDiet + Nuts	Yes	1
4	1111	MedDiet + VOO	No	0
...	...	...	...	...
6319	120	Control	No	0
6320	118	Control	No	0
6321	351	MedDiet + Nuts	No	0
6322	499	MedDiet + VOO	No	0
6323	1257	MedDiet + Nuts	No	0

6324 rows x 4 columns

```
df_patients.groupby('group')['event_int'].sum()
```

split

by diet group

apply

sum() of event counts

combine

	patient-id	group	event	event_int
0	436	Control	Yes	1
1	1130	Control	No	0
2	1131	MedDiet + VOO	No	0
3	1132	MedDiet + Nuts	Yes	1
4	1111	MedDiet + VOO	No	0
...	...	...	...	...
6319	120	Control	No	0
6320	118	Control	No	0
6321	351	MedDiet + Nuts	No	0
6322	499	MedDiet + VOO	No	0
6323	1257	MedDiet + Nuts	No	0

6324 rows x 4 columns

Hands-on

- Pt1: Work with the dataset from the previous exercise to answer the question: Is the mediterranean diet associated with less cardiovascular events?
- Pt2: Work with the `predimed_smokers.csv` dataset to answer the question: What is the mean age per diet group?

Exercise pt 1: 15 minutes



```
exercises/tabular_split  
_apply_combine/split_ap  
ply_combine_part1.ipynb
```

Exercise pt 2: 7 minutes



```
exercises/tabular_split  
_apply_combine/split_ap  
ply_combine_part2.ipynb
```

Messy data

Variables are
stored in both
rows and columns



city	type	date	temperature
Bilbao	tmax	2024-07-03	34
Bilbao	tmin	2024-07-03	25
Bordeaux	tmax	2024-03-21	29
Bordeaux	tmin	2024-03-21	23
Berlin	tmax	2021-08-16	21
Berlin	tmin	2021-08-16	14
Heraklion	tmax	2021-09-01	30
Heraklion	tmin	2021-09-01	23

Messy data

Variables are
stored in both
rows and columns



city	type	date	temperature
Bilbao	tmax	2024-07-03	34
Bilbao	tmin	2024-07-03	25
Bordeaux	tmax	2024-03-21	29
Bordeaux	tmin	2024-03-21	23
Berlin	tmax	2021-08-16	21
Berlin	tmin	2021-08-16	14
Heraklion	tmax	2021-09-01	30
Heraklion	tmin	2021-09-01	23

Column headers
are values, not
variable names



subject	date	A	B
PB	2024-07-03	0.12	0.19
VM	2024-03-21	0.37	0.41
TZ	2021-08-16	0.68	0.73
LS	2021-09-01	0.07	0.08
ZS	2023-11-11	0.08	0.16

Messy data

Variables are stored in both rows and columns



city	type	date	temperature
Bilbao	tmax	2024-07-03	34
Bilbao	tmin	2024-07-03	25
Bordeaux	tmax	2024-03-21	29
Bordeaux	tmin	2024-03-21	23
Berlin	tmax	2021-08-16	21
Berlin	tmin	2021-08-16	14
Heraklion	tmax	2021-09-01	30
Heraklion	tmin	2021-09-01	23

Column headers are values, not variable names



subject	date	A	B
PB	2024-07-03	0.12	0.19
VM	2024-03-21	0.37	0.41
TZ	2021-08-16	0.68	0.73
LS	2021-09-01	0.07	0.08
ZS	2023-11-11	0.08	0.16

Some variables are stored in the file names



2024-01_prices_DE.csv
2024-01_prices_FR.csv
2024-02_prices_DE.csv
2024-02_prices_FR.csv

Messy data

Variables are stored in both rows and columns 🤯

city	type	date	temperature
Bilbao	tmax	2024-07-03	34
Bilbao	tmin	2024-07-03	25
Bordeaux	tmax	2024-03-21	29
Bordeaux	tmin	2024-03-21	23
Berlin	tmax	2021-08-16	21
Berlin	tmin	2021-08-16	14
Heraklion	tmax	2021-09-01	30
Heraklion	tmin	2021-09-01	23

Column headers are values, not variable names 🤯

subject	date	A	B
PB	2024-07-03	0.12	0.19
VM	2024-03-21	0.37	0.41
TZ	2021-08-16	0.68	0.73
LS	2021-09-01	0.07	0.08
ZS	2023-11-11	0.08	0.16

Some variables are stored in the file names 🤯

2024-01_prices_DE.csv
2024-01_prices_FR.csv
2024-02_prices_DE.csv
2024-02_prices_FR.csv

Multiple variables are stored in one column 🤯

country	year	variable	cases
Angola	2000	sp_m_014	186.0
Angola	2001	sp_m_014	230.0
Angola	2002	sp_m_014	435.0
Angola	2003	sp_m_014	409.0
Angola	2004	sp_m_014	554.0
Angola	2005	sp_m_014	520.0
Angola	2006	sp_m_014	540.0

Messy data

“Tidy datasets are all alike but every messy dataset is messy in its own way”
– Hadley Wickham

Variables are stored in both rows and columns 🤪

city	type	date	temperature
Bilbao	tmax	2024-07-03	34
Bilbao	tmin	2024-07-03	25
Bordeaux	tmax	2024-03-21	29
Bordeaux	tmin	2024-03-21	23
Berlin	tmax	2021-08-16	21
Berlin	tmin	2021-08-16	14
Heraklion	tmax	2021-09-01	30
Heraklion	tmin	2021-09-01	23

Column headers are values, not variable names 🤪

subject	date	A	B
PB	2024-07-03	0.12	0.19
VM	2024-03-21	0.37	0.41
TZ	2021-08-16	0.68	0.73
LS	2021-09-01	0.07	0.08
ZS	2023-11-11	0.08	0.16

Some variables are stored in the file names 🤪

2024-01_prices_DE.csv
2024-01_prices_FR.csv
2024-02_prices_DE.csv
2024-02_prices_FR.csv

Multiple variables are stored in one column 🤪

country	year	variable	cases
Angola	2000	sp_m_014	186.0
Angola	2001	sp_m_014	230.0
Angola	2002	sp_m_014	435.0
Angola	2003	sp_m_014	409.0
Angola	2004	sp_m_014	554.0
Angola	2005	sp_m_014	520.0
Angola	2006	sp_m_014	540.0

Tidy Data

Tidy data

In tidy data:

1. Each variable forms a column
2. Each observation forms a row
3. Each type of observational unit forms a table

Variables (or features, attributes)

Observations
(or samples)

Subject ID	Condition ID	Date	Response time (ms)	Response
VM	732	7.3.2024	28	LEFT
VM	732	8.3.2024	41	RIGHT
PB	665	9.3.2024	73	LEFT

Variables increase when new types of measurements are introduced

Observations increase when new units (dates, subjects, ...) are measured

Hands on:

Same data, different organization

Which one is best for data analysis?

	John Smith	Jane Doe	Mary Johnson
treatmenta	—	16	3
treatmentb	2	11	1

name	trt	result
John Smith	a	—
Jane Doe	a	16
Mary Johnson	a	3
John Smith	b	2
Jane Doe	b	11
Mary Johnson	b	1

	treatmenta	treatmentb
John Smith	—	2
Jane Doe	16	11
Mary Johnson	3	1

Hands on:

Same data, different organization

Which one is best for data analysis?

What do we want?

**We want data to be in a natural format,
such that we can make use of the
optimizations already built in the library
(as we have already seen)**

treatmenta
treatment

tb
2
11
1

Jane Doe b 11
Mary Johnson b 1

Back to the exercise

What is wrong with this data?

	patient-id	group	sex_age	smoke_never	smoke_former	smoke_current	event
0	1325	MedDiet + VOO	Male_72	0	0	1	0
1	1326	Control	Female_69	1	0	0	0
2	1327	MedDiet + Nuts	Female_64	0	1	0	0
3	1328	Control	Female_70	1	0	0	0
4	1329	MedDiet + VOO	Male_65	0	1	0	0
...	...	...	...	...	...	...	...
395	1720	MedDiet + VOO	Male_67	0	0	1	0
396	1721	Control	Male_68	0	1	0	0
397	1722	MedDiet + VOO	Female_61	1	0	0	0
398	1723	MedDiet + VOO	Female_72	1	0	0	0
399	1724	MedDiet + Nuts	Female_70	1	0	0	0

400 rows x 7 columns

Pt2: Work with the
`predimed_smokers.csv`
dataset to answer the question:
What is the mean age per diet group?

Exercise pt 2: 7 minutes



`exercises/tabular_split
_apply_combine/split_ap
ply_combine_part2.ipynb`

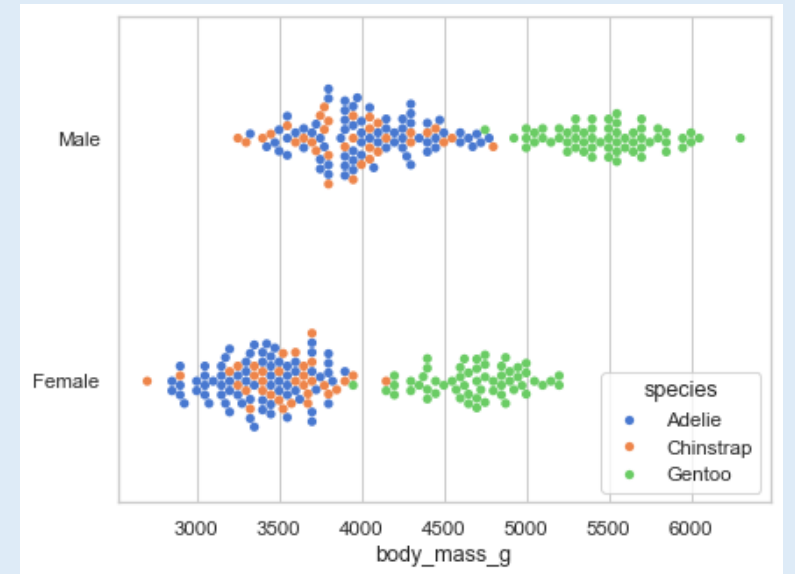
	patient-id	group	sex_age	smoke_never	smoke_former	smoke_current	event
0	1325	MedDiet + VOO	Male_72	0	0	1	0
1	1326	Control	Female_69	1	0	0	0
2	1327	MedDiet + Nuts	Female_64	0	1	0	0
3	1328	Control	Female_70	1	0	0	0
4	1329	MedDiet + VOO	Male_65	0	1	0	0
...	...	...	...	...	...	...	...
395	1720	MedDiet + VOO	Male_67	0	0	1	0
396	1721	Control	Male_68	0	1	0	0
397	1722	MedDiet + VOO	Female_61	1	0	0	0
398	1723	MedDiet + VOO	Female_72	1	0	0	0
399	1724	MedDiet + Nuts	Female_70	1	0	0	0

400 rows × 7 columns

Why is tidy data good?

- Many analyses require a simple sequence of steps:
 - Filter by individual variables to discard data that is not needed
 - Group and summarize
 - Re-arrange (e.g. sort)
 - Visualize
- Joining tidy tables is easy!
- One can write generic code that takes tidy data as input.
For example, **seaborn** relies on tidy data to make complex plots

```
sns.swarmplot(  
    data=df,  
    x="body_mass_g",  
    y="sex",  
    hue="species",  
)
```



Common operations on tabular data

- Combine information across tables (**join**, **anti-join**)
 - **Join**: e.g., combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
 - **Anti-join**: e.g. student compiles list of outliers, exclude them from the table of experiments to analyze

Common operations on tabular data

- Combine information across tables (**join, anti-join**)
 - **Join**: e.g., combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
 - **Anti-join**: e.g. student compiles list of outliers, exclude them from the table of experiments to analyze
- Summary tables (**split-apply-combine**)
 - E.g., compute average measurement and standard deviation by experimental condition and treatment dosage

Common operations on tabular data

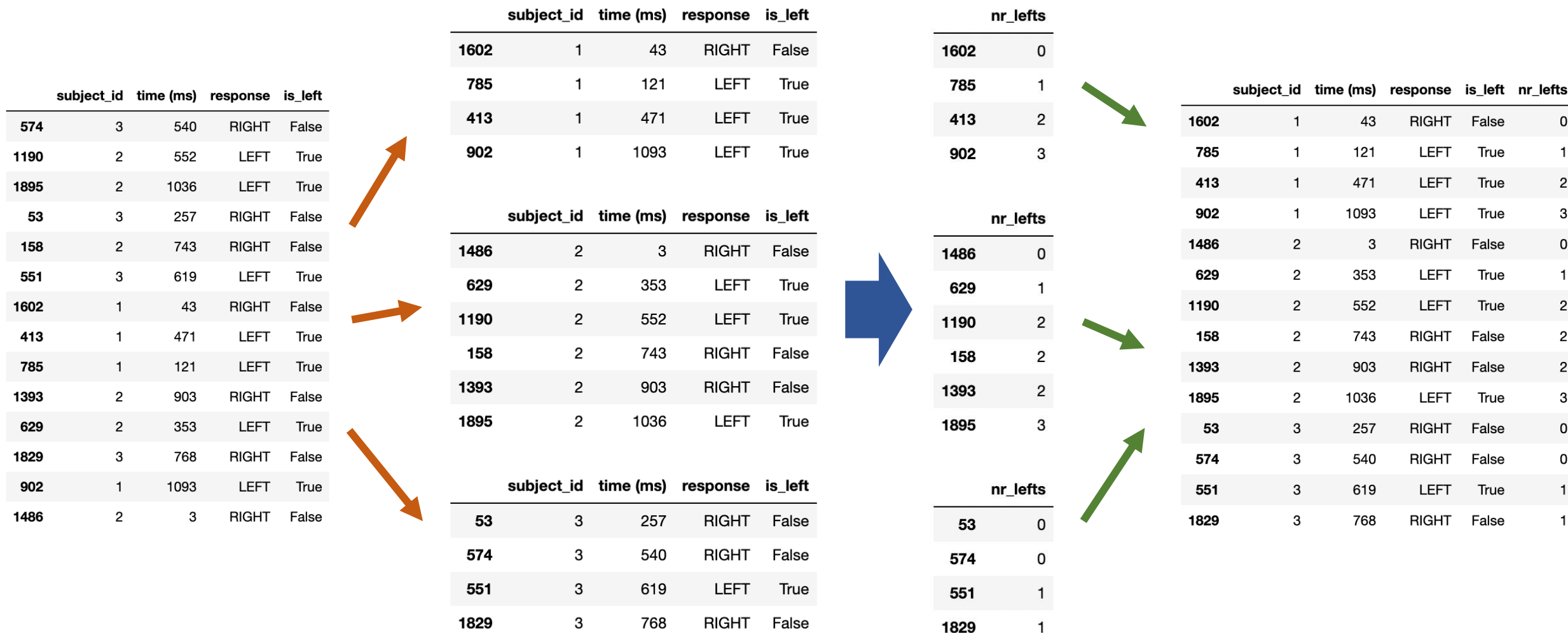
- Combine information across tables (**join, anti-join**)
 - **Join**: e.g., combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
 - **Anti-join**: e.g. student compiles list of outliers, exclude them from the table of experiments to analyze
- Summary tables (**split-apply-combine**)
 - E.g., compute average measurement and standard deviation by experimental condition and treatment dosage

Common operations on tabular data

- Combine information across tables (**join, anti-join**)
 - **Join**: e.g., combine table with experiments results with table with experiments metadata (date, location, experimenter, free-form notes, ...)
 - **Anti-join**: e.g. student compiles list of outliers, exclude them from the table of experiments to analyze
- Summary tables (**split-apply-combine**)
 - E.g., compute average measurement and standard deviation by experimental condition and treatment dosage
- **Window functions** to vectorize complex computations over groups
 - E.g., compute some operation considering more than one row at the time

Window functions operations

```
df['nr_lefts'] = df.sort_values('time (ms)').groupby('subject_id')['is_left'].cumsum()
```



Common operations on tabular data

- Combine information across tables (**join, anti-join**)

- ~~Join: e.g. combine table with experiments results with table with experiments metadata~~

- Libraries are optimized to operate on tabular data

- ~~Anti~~

- ~~exp~~

- Summ

- E.g.

- ~~and~~

- The goal of today was to introduce some of these vectorized operations and why it is important to use them

- ~~condition~~

- **Window functions** to vectorize complex computations over groups

- E.g., compute some operation considering more than one row at the time

Global summary

- There are many different data structures, each specialized in efficiently processing one type of data.
- Code performance grows differently with data size: Big-O
- NumPy array efficiently store data in a C-native memory block, interpreted as an array using some metadata.
- NumPy operations that only need to change the metadata do so, creating a view of the same memory block. These operations are $O(1)$!
- Tabular data can also be vectorized using joins, anti-joins, split-apply-combine operations, and window functions.
- For these operations to be efficient and painless, data should be stored in a tidy data format.

Dealing with changes in the data

- Recommendations:
 - NEVER overwrite a data file. Treat data files as immutable
 - Use versioning for changes in the data file, and load the latest version for new analyses, old versions to reproduce previous results
 - (pond is a library Pietro is working on to automatize this process)
- Like in computer code:
 - Adding new columns / rows is generally okay
 - Deleting/changing a column is not! Code will break! Add a new column instead

Thank you!



